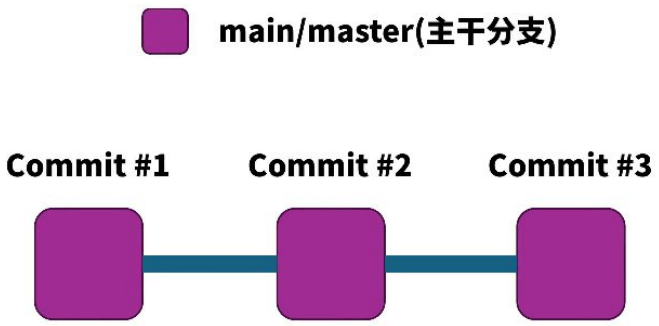
-- 以下为AI生成的图文笔记内容 --

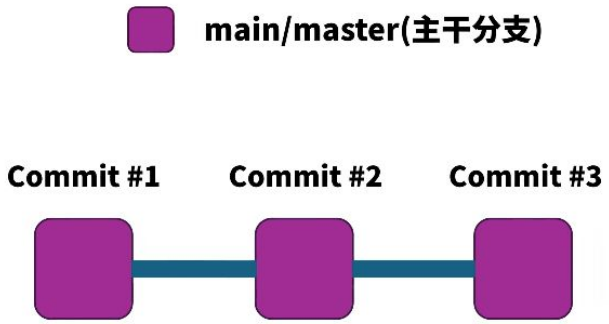
一、*分支概念00:00*



本节课我们来看一个Git的重要概念

-
- **定义**：分支是存储库的不同版本，每个仓库默认有一个main分支或master分支（主干分支）
- **作用**：允许在不影响主干的情况下进行独立开发

1. 主干分支 00:03

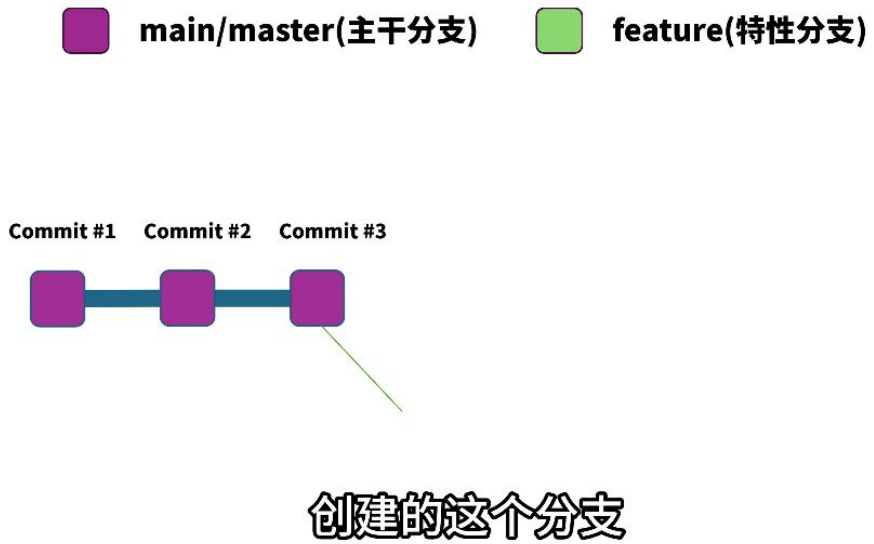


比如我从main创建了一个叫feature的分支

-
- **默认分支**：每个Git仓库初始时自动创建main/master分支
- **特性**：

- 作为项目的基础版本
- 通常存放稳定可发布的代码
- 其他分支最终都需要合并回主干

2. 特性分支 00:15



-
- **创建方式**：从main分支创建副本（如feature分支）
- **工作特点**：
  - 创建时是主干分支的快照，初始代码与主干完全一致
  - 修改仅对当前分支有效，不影响其他分支
  - 不同分支的修改相互隔离
- **协作优势**：
  - 多人可同时创建各自分支独立开发
  - 避免直接修改主干造成冲突

3. 学习Git分支 01:05

-

- **演示工具**：通过learnGitBranching.js.org网站动画演示分支操作
- **典型流程**：
  - 从main创建feature分支
  - 在feature分支提交多次
  - 遇到紧急bug时切换回main创建bugfix分支
  - 修复完成后合并回main
  - 返回feature继续开发

1）分支创建与提交 01:23

-

- **创建命令**：git checkout -b <分支名>
- **提交操作**：
  - 星号(*)标记当前所在分支
  - 每次提交会在当前分支生成新节点
  - 未完成的修改可保留在特性分支

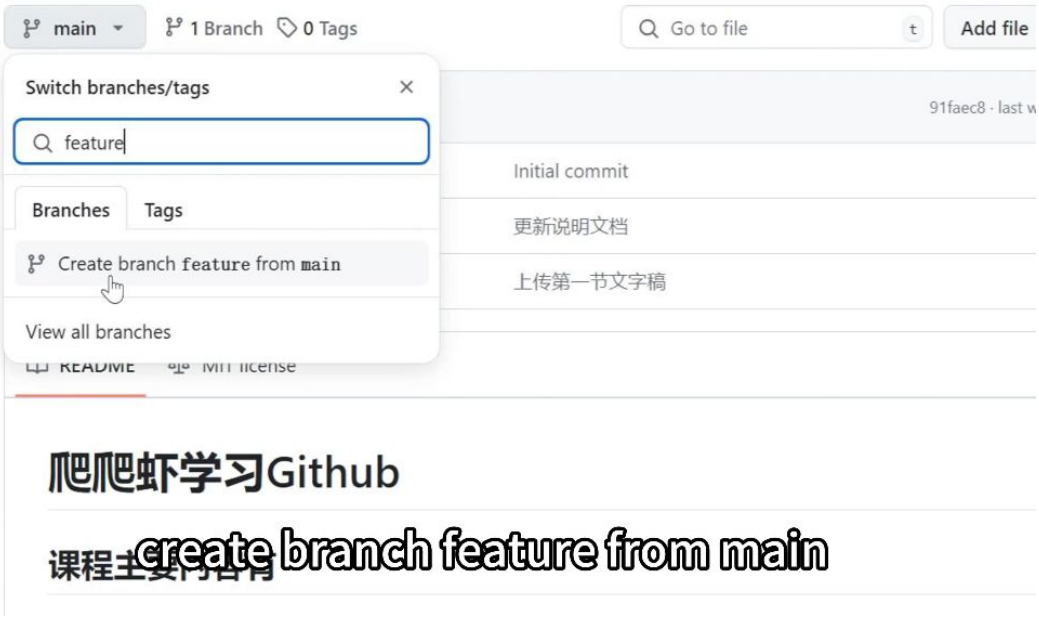
2）bug分支创建与合并 01:50

-

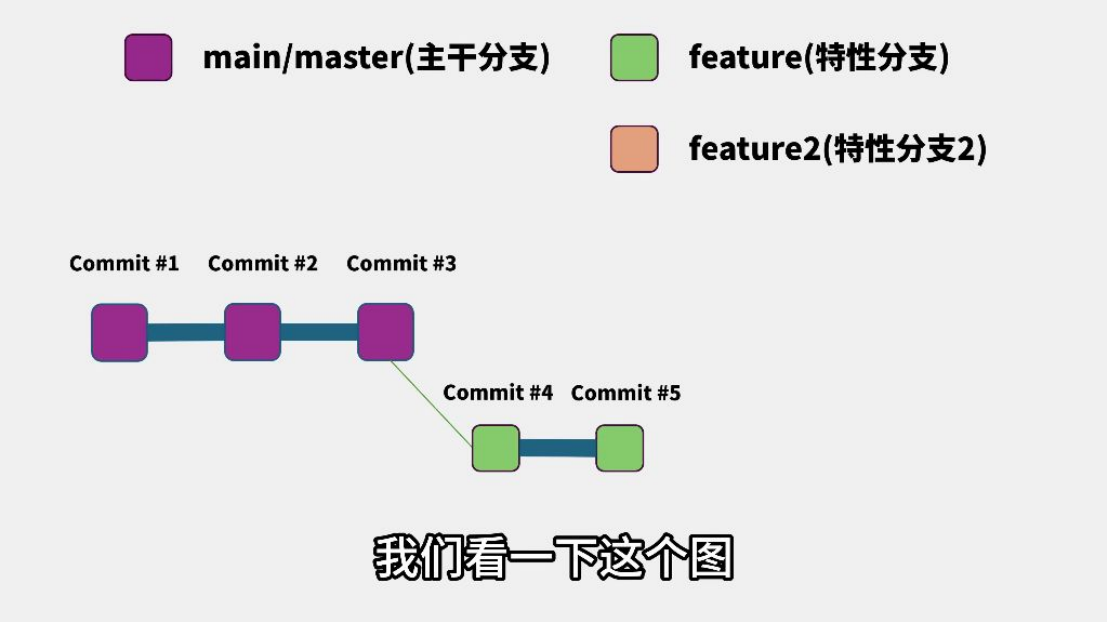
- **紧急修复流程**：
  - 切换回main分支：git switch main

    - 创建bugfix分支
    - 提交修复代码
    - 合并到main：git merge bugfix --no-ff
    - 切换回feature继续开发
- **最终合并**：功能开发完成后将feature分支合并回main

4. GitHub网站创建分支 02:55

- 

- **创建方法**：
    - 点击分支切换按钮
    - 输入新分支名（如feature）
    - 选择基于哪个分支创建（默认main）
    - 点击"Create branch"按钮
- **修改提交**：
    - 在分支内修改文件后提交
    - 修改仅存在于当前分支
    - 需要合并(merge)才能使修改出现在main分支

- 

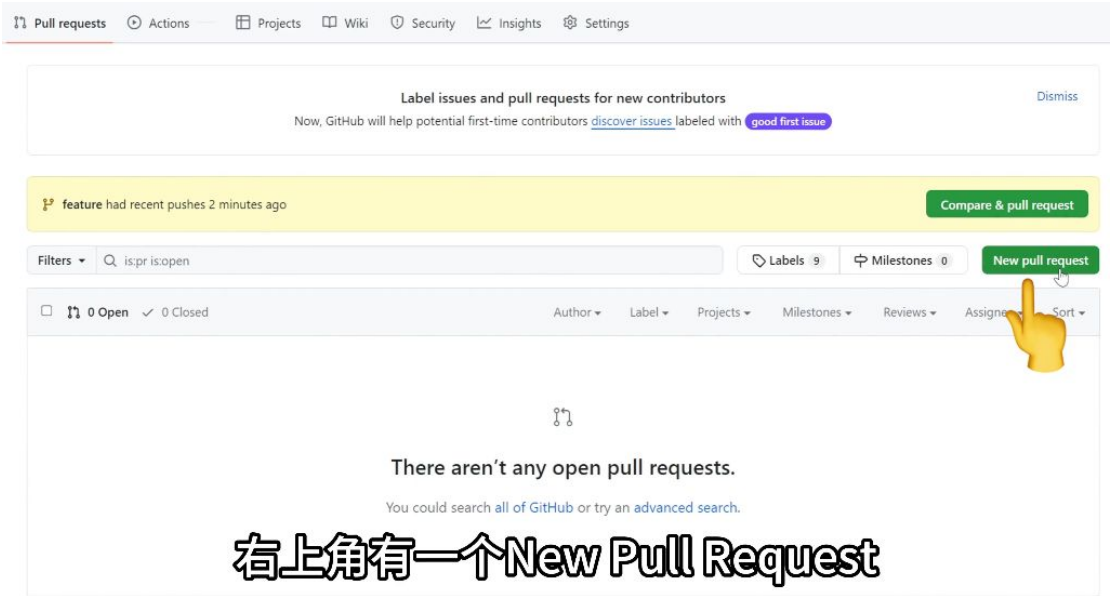
- **分支继承关系**：
  - 基于feature创建的分支包含feature和main的所有代码
  - 基于main创建的分支只能看到main的代码
- **分支管理**：
  - 通过Branches页面查看所有分支状态
  - 可基于任意现有分支创建新分支

二、*合并请求* ***04:49***

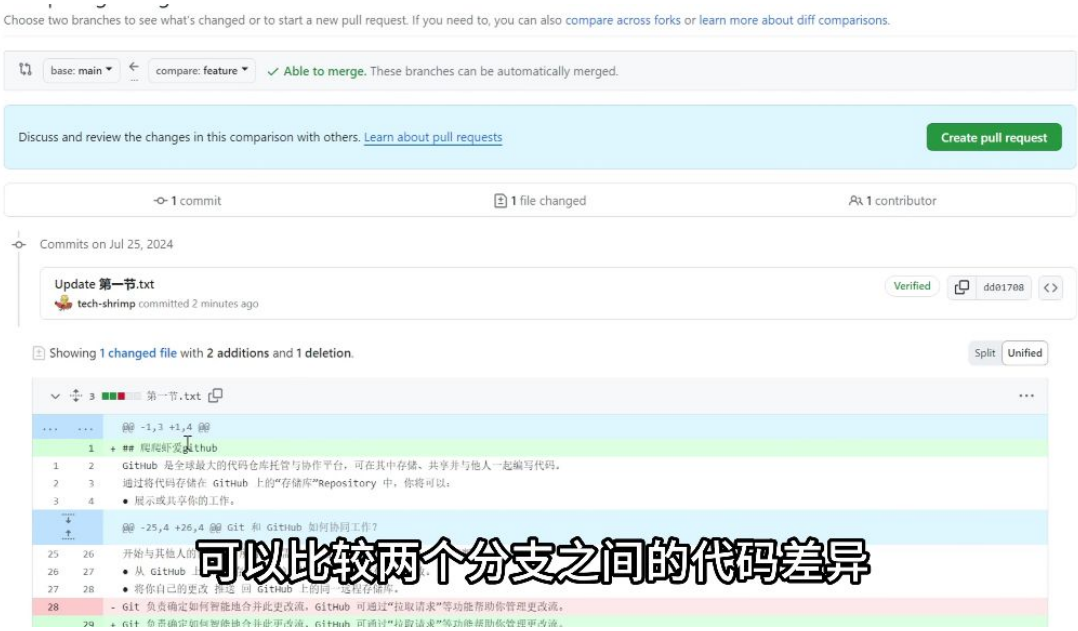
-

- **定义**：Pull Request（PR）中文翻译为"拉取请求"或"合并请求"，是GitHub中用于将更改从一个分支合并到另一个分支的提案机制。
- **核心功能**：比较两个分支间的代码差异，并允许仓库管理员审核代码改动。

1. 创建合并请求 05:42

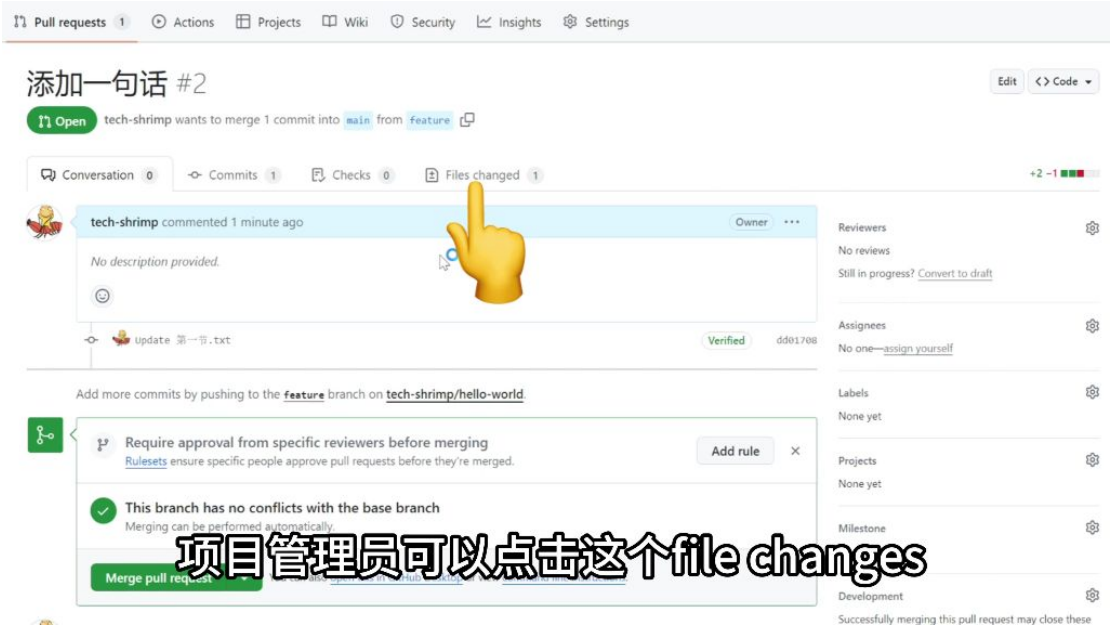


-
- **操作步骤**：
  - **入口**：在GitHub网页版点击"Pull requests"选项卡
  - **新建**：点击右上角"New Pull Request"按钮
  - **方向选择**：通常选择从功能分支(feature)向主干分支(main)合并
  - **差异比较**：系统会自动显示两个分支间的代码差异
  - **提交信息**：需要填写PR标题和详细描述
  - **创建确认**：点击"Create Pull Request"完成创建

- 

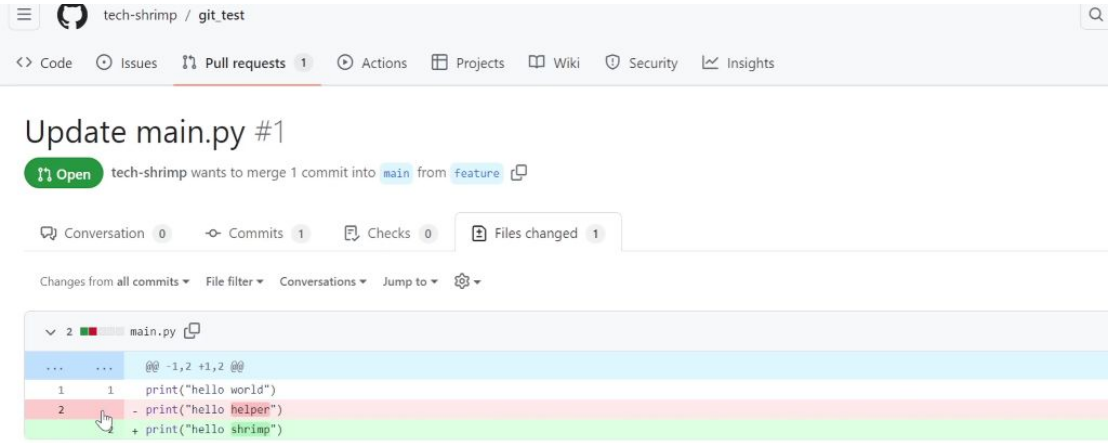
- **关键要素**：
  - **合并状态**：显示"可自动合并"或存在冲突
  - **变更统计**：显示提交次数、文件变更数量
  - **贡献者**：显示参与本次PR的贡献者数量

2. 审核代码 06:34

- 

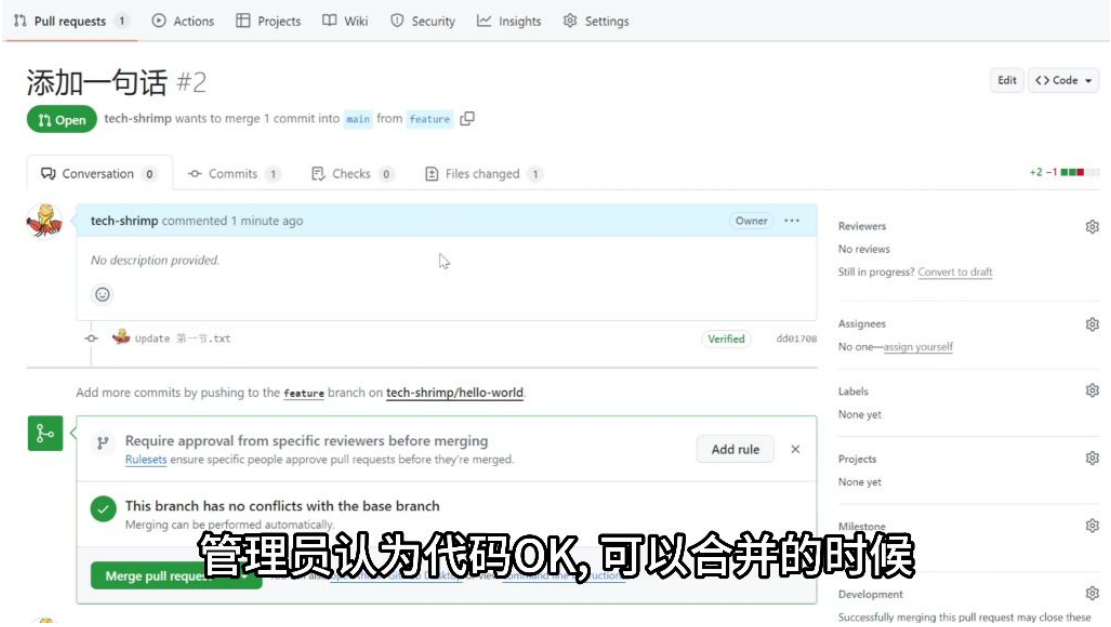
- **审核流程**：
  - **入口**：管理员点击"Files changed"开始审核
  - **行级评论**：可在特定代码行点击"+"号添加评论

- **审核结论**：有三种可选意见：
  - **Comment**：仅评论不决定合并
  - **Approve**：同意代码合并
  - **Request changes**：要求修改后重新审核



项目管理员审核代码的时候 可以在这个页面

-
- **审核工具**：
  - **差异高亮**：绿色表示新增内容，红色表示删除内容
  - **文件筛选**：可过滤查看特定文件的变更
  - **版本对比**：支持查看所有提交的完整变更历史



管理员认为代码OK，可以合并的时候

- **合并执行**：
  - **条件**：当所有问题修改完成且管理员确认后
  - **操作**：点击"Merge pull request"按钮
  - **结果验证**：合并后可在主干分支查看已合并的代码
  - **分支清理**：建议删除已完成合并的功能分支保持仓库整洁

三、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
Git分支概念	分支是存储库的不同版本，默认每个仓库有main/master主干分支，创建分支即创建副本（如feature分支）。分支间代码修改互不影响。	**分支独立性**：修改仅限当前分支，切换分支后不可见其他分支的改动。	★★
分支创建与切换	命令示例：git checkout -b feature（创建并切换分支）、git switch main（切换回主干）。网页版（GitHub）通过分支按钮直接创建。	**分支来源**：可基于main或其他分支（如feature2基于feature会继承其代码）。	★★
分支合并流程	功能开发完成后需合并回主干（merge）。**典型场景**：修复生产环境bug时，需切回main创建	**合并顺序**：需先处理紧急bug，再回归功能开发。	★★★

	bugfix分支，修复后合并回主干，再继续开发功能分支。		
Pull Request（PR)	合并请求提案，用于将分支代码合并到主干。流程：创建PR → 代码差异对比 → 管理员审核（code review） → 批准（approve）或要求修改（request changes）。	**审核意见类型**：comment（仅评论）、approve（同意合并）、request changes（需修改）。	★★★★
分支管理最佳实践	合并后删除已合并的功能分支（如GitHub网页端操作），保持分支树清晰。	**误删风险**：确保分支代码已合并再删除。	★★